Python Engineering - Class 04 Reference Guide

Functions and Decomposition

Reference guide to the syntax and tools used in this class. It explains what each element does, how it is written and when it is useful.

Quick map

Concept	Syntax	Idea
def	def name(parameters):	Defines a function object.
Parameters	def f(x, y):	Names used inside the function to receive information.
Arguments	f(2, 3)	Concrete values supplied during a call.
return	return value	Sends a result back to the caller and ends the function execution.
Local scope	local_var	Names created inside a function normally exist only during that call.
Default arguments	def f(g=9.81):	Provide a value when the caller omits an argument.
Multiple return values	return h, v, t	Python packs several returned values into a tuple.
Boolean guard	if not valid: return ...	Stops invalid processing early and makes control flow explicit.
Function call	result = f(...)	Parentheses execute the function; the returned value can be stored.

1. def

Syntax: `def name(parameters):`

Defines a function object.

```
def dynamic_pressure(rho, speed):
    ...
```

Control question: before running the example, predict the result type and shape.

2. Parameters

Syntax: `def f(x, y):`

Names used inside the function to receive information.

```
def euler_step(h, v, t, g, dt):
    ...
```

Control question: before running the example, predict the result type and shape.

3. Arguments

Syntax: `f(2, 3)`

Concrete values supplied during a call.

```
h_new, v_new, t_new = euler_step(h, v, t, g, dt)
```

Control question: before running the example, predict the result type and shape.

4. return

Syntax: `return value`

Sends a result back to the caller and ends the function execution.

```
return h_new, v_new, t_new
```

Control question: before running the example, predict the result type and shape.

5. Local scope

Syntax: `local_var`

Names created inside a function normally exist only during that call.

```
def f(x):
    local_value = x**2
    return local_value
```

Control question: before running the example, predict the result type and shape.

6. Default arguments

Syntax: `def f(g=9.81):`

Provide a value when the caller omits an argument.

```
def initial_energy(h0, v0, g=9.81):
    ...
```

Control question: before running the example, predict the result type and shape.

7. Multiple return values

Syntax: `return h, v, t`

Python packs several returned values into a tuple.

```
return max_h, v_f, t_f
```

Control question: before running the example, predict the result type and shape.

8. Boolean guard

Syntax: `if not valid: return ...`

Stops invalid processing early and makes control flow explicit.

```
if not validate_inputs(...):
    return None
```

Control question: before running the example, predict the result type and shape.

9. Function call

Syntax: `result = f(...)`

Parentheses execute the function; the returned value can be stored.

```
result = dynamic_pressure(1.225, 100.0)
```

Control question: before running the example, predict the result type and shape.

Common mistakes

- Confusing a function name with a function call using parentheses.
- Not checking types, indices or shapes before an operation.
- Using a tool because it works without being able to explain what it receives and returns.

Mental cheat sheet

```
def: def name(parameters):
Parameters: def f(x, y):
Arguments: f(2, 3)
return: return value
Local scope: local_var
Default arguments: def f(g=9.81):
Multiple return values: return h, v, t
Boolean guard: if not valid: return ...
Function call: result = f(...)
```